

VII: Arbres de décision, Forêts aléatoires et Gradient Boosting

Guillaume Guex guillaume.guex@unil.ch

Les arbres de décision

Les **arbres de décision** sont des algorithmes de ML **supervisés**.

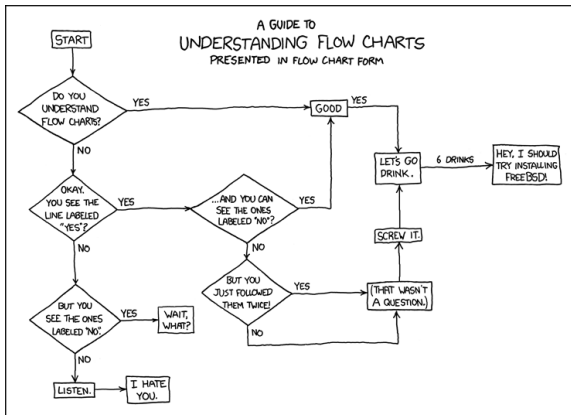
- Existent en version **classification** et **régression**.
- Sont très faciles à **interpréter** (white box).
- Sont **non-paramétriques**.
- Très **rapides dans les prédictions**.

Malheureusement, en tant que tels, ils sont fréquemment sujet à la **sur-paramétrisation** ou **font des prédictions approximatives**.

Cependant, ils servent de fondation pour de **nombreux algorithmes dérivés très performants**.

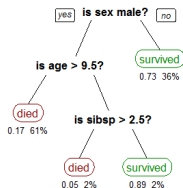
Analogie : les flowcharts

Dans la culture populaire, les **flowcharts** sont des objets qui s'apparentent à des arbres de décision (mais ces flowcharts ne sont pas forcément des arbres).



Exemple d'arbre entraîné

L'entraînement d'un **arbre de classification** sur un jeu de données (Titanic) résulte en un arbre de la sorte :



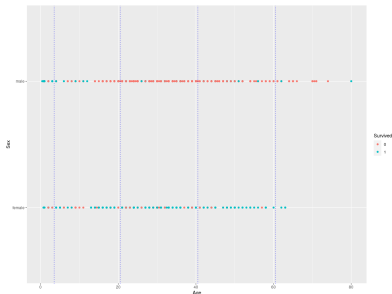
Source : Stephen Milborrow CC BY-SA 3.0 via Wikimedia Commons

- Le nœud du haut s'appelle la **racine**.
- Les nœuds qui séparent le flux en deux sont les **noeuds de décision** (via une variable numérique ou binaire).
- Les nœuds terminaux, qui attribuent les classes, sont les **feuilles**.

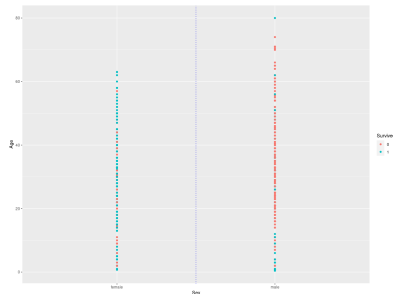
Entrainement d'un arbre de classification

Pour obtenir un arbre de la sorte l'**entrainement** se passe de manière itérative. On **commence par la racine**.

On va lister toutes les **séparations possibles sur toutes les variables**.



Variables numériques : **entre chaque paire de points contigus**.



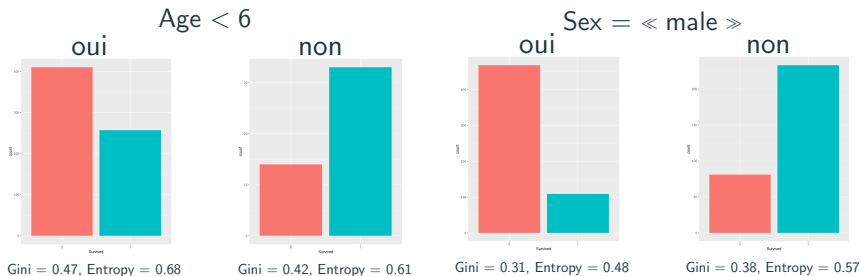
Variables binaires : en **deux groupes**.

Entrainement d'un arbre de classification

On peut utiliser deux **mesures de variété** au sein des groupes :

- L'**indice de Gini** : $1 - \sum_c p_c^2$
- L'**entropie** : $-\sum_c p_c \log(p_c)$

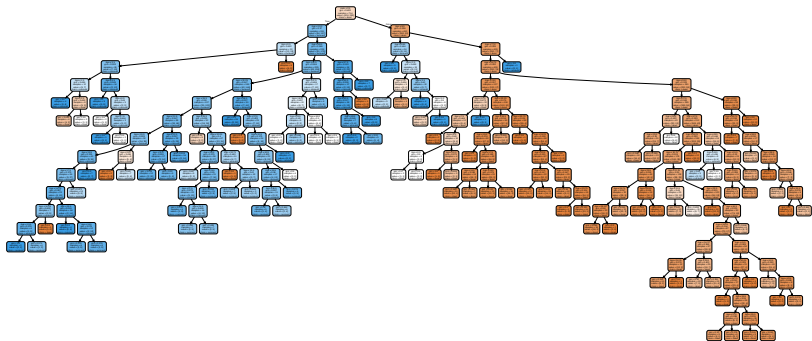
où c sont les classes et p_c leur proportion dans le groupe.



On retient la séparation qui donne **le plus petit score moyen pondéré**.

Entrainement d'un arbre de classification

L'algorithme procède de la même manière pour chaque feuille obtenue. L'algorithme s'arrête lorsque toutes les feuilles sont **pures** (ne contiennent plus qu'une classe) ou que les individus qui la composent sont **indiscernables** (selon leurs variables d'entrée).



En procédant ainsi, le risque de **sur-paramétrisation** est énorme.

Limiter la sur-paramétrisation

Plusieurs stratégies sont possibles pour limiter la taille de l'arbre, et ainsi son risque de sur-paramétrisation :

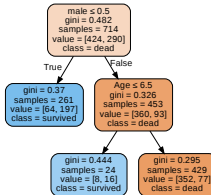
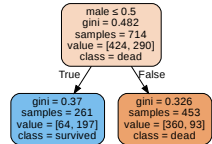
- Limiter la **profondeur** de l'arbre.
- Limiter le **nombre de feuilles**.
- Poser une **taille minimale** en dessous de laquelle les groupes ne sont plus séparés.
- **Élaguer (to prune)** l'arbre complet.

La procédure d'élagage se fait généralement en sélectionnant le **sous-arbre** T qui minimise :

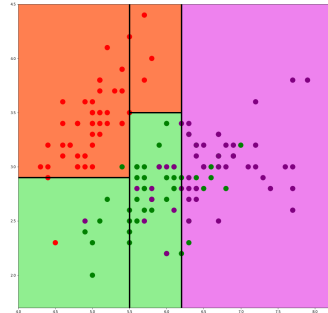
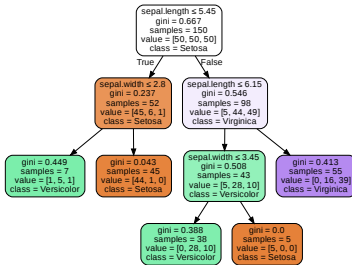
$$R_{\alpha}(T) = R(T) + \alpha L(T)$$

où $R(T)$ est l'**erreur de classification** de l'arbre et $L(T)$ le nombre de feuilles. L'**hyperparamètre** α contrôle la régularisation.

Élagage de l'arbre

 $\alpha = 0.002$  $\alpha = 0.005$  $\alpha = 0.05$ 

Les **frontières de décisions** d'un arbre sont des hyperplans qui segmentent les sous-espaces restants de manière séquentielle.



Les **arbres de régression** se basent sur le même principe, mais :

- Une feuille G attribue une **valeur moyenne** $\bar{y}_G$ aux individus qu'elle contient :

$$\bar{y}_G = \frac{1}{n_G} \sum_{i \in G} y_i.$$

- Pour évaluer une séparation, on utilise généralement l'**erreur carrée** des groupes :

$$E(G) = \sum_{i \in G} (y_i - \bar{y}_G)^2$$

La **moyenne pondérée sur les groupes** permet de comparer les séparations entre elles.

Demo : https://arogozhnikov.github.io/2016/06/24/gradient_boosting_explained.html

Les forêts aléatoires

Les arbres de décisions souffrent souvent des défauts suivants :

- Les arbres profonds sont **sur-paramétrisés**.
- Les arbres peu profonds sont des **weak learners** (peu de pouvoir prédictif).
- Ils sont très **sensibles** aux données d'entraînement.

C'est pourquoi on les utilise plutôt dans des méthodes d'**apprentissage ensembliste**.

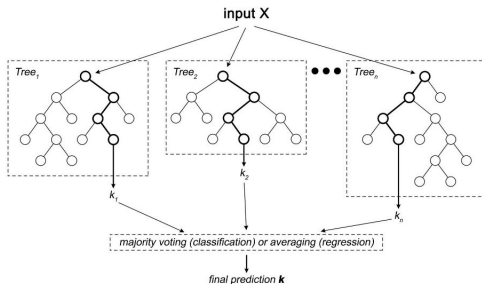
Le principe de l'apprentissage ensembliste est d'utiliser de **nombreux modèles de prédiction** et de **combiner leur résultat pour obtenir une prédiction finale**, plus pertinente.

Les **forêts aléatoires** et le **gradient boosting** sont des exemples de méthode d'apprentissage ensembliste qui utilisent des arbres.

Les forêts aléatoires

Le principe des **forêts aléatoires** est de :

- **Générer aléatoirement m arbres** à partir du jeu de données.
- Faire des prédictions avec la **majorité (classification)** ou la **moyenne (régression)** des résultats.

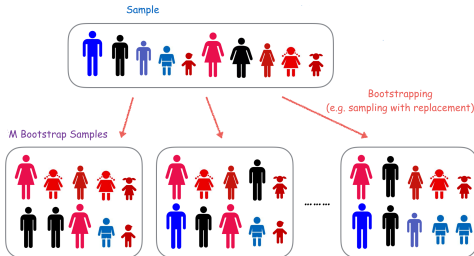


Source : <https://www.quora.com/Is-random-forest-an-ensemble-method>

Deux mécanismes sont à l'œuvre pour générer des arbres aléatoires.

Bootstrapping

Le premier consiste à générer m jeux de données par **bootstrapping** sur le jeu d'entraînement de taille n . On va tirer n **individus aléatoirement avec remise** du jeu de données initial pour générer m nouveaux jeux de données.



Source : https://dgarcia-eu.github.io/SocialDataScience/2_SocialDynamics/025_Bootstrapping/Bootstrapping.html

Chaque nouveau jeu de données servira à entraîner un seul arbre.

Sélection aléatoire des variables

Le second mécanisme consiste à utiliser uniquement un **sous-ensemble aléatoire de $q < p$ variables** pour l'entraînement de chaque arbre. Généralement, on pose q comme $\sqrt{p}$ ou $\log(p)$.

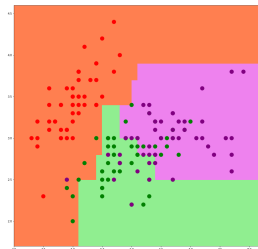
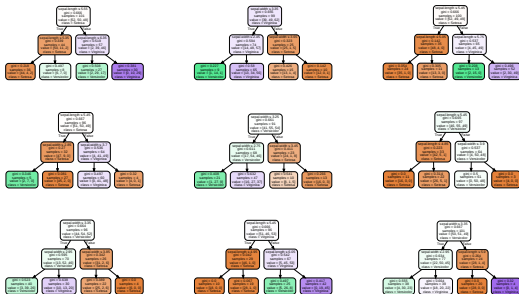
Une approximation de l'erreur de la forêt peut être estimée via les individus qui n'ont pas été sélectionnés lors du bootstrapping. On appelle ces individus l'échantillon **out-of-bag**.

Pour résumé :

- Le bootstrapping **réduit la sensibilité aux données**.
- Le tirage des variables **augmente la variété des arbres construits**.

Le processus qui consiste à entraîner plusieurs modèles par bootstrapping et d'agréger leur prédiction s'appelle le **bagging**.

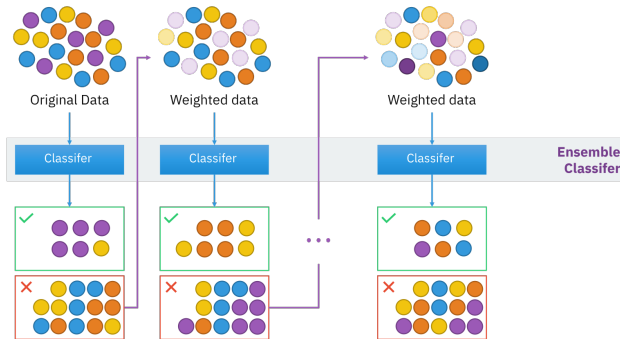
Exemple : les iris



Le gradient boosting

Le boosting

Le **boosting** est une pratique qui fait partie de l'apprentissage ensembliste. Le principe est de créer les différents modèles de l'ensemble d'une manière **séquentielle** en utilisant à chaque fois les **erreurs du modèle précédent** pour modifier l'entraînement.



Il existe de nombreux algorithmes de boosting. En voici les principaux dans l'ordre d'apparition :

- **AdaBoost** : le premier.
- **Gradient Boost** : le théorique.
- **XGBoost** : le performant.
- **Light Gbm** : l'optimisé.

Il en existe encore de nombreux autres. Les algorithmes de boosting sont très performants, en particulier pour les **jeux de données de taille modérée**.

Le gradient boosting

Le **gradient boosting** est un algorithme très général, qui peut s'utiliser avec n'importe quel classificateur ou régresseur.

L'algorithme se base sur la minimisation d'une **fonction de perte donnée** $L(y, \hat{y})$ entre une prédiction $\hat{y}$ et la vraie valeur y .

Nous allons voir comment l'utiliser avec des arbres de régression sur un exemple de données simple (but : prédire Y avec X_1, X_2, X_3) :

X_1	X_2	X_3	Y
2	51	0.3	13
3	58	0.1	14
7	45	0.5	18
8	61	0.8	15
11	48	0.7	20

Le gradient boosting : initialisation

L'initialisation consiste à trouver une **valeur fixe** $\hat{y}^{(0)}$ pour la variable Y qui **minimise la somme des pertes** $\sum_i L(y_i, \hat{y}^{(0)})$.

Pour une régression, on prends généralement l'**erreur au carrée** $L(y, \hat{y}) = (y - \hat{y})^2/2$, ce qui implique que $\hat{y}^{(0)}$ **est la moyenne**.

X_1	X_2	X_3	Y	$\hat{y}^{(0)}$
2	51	0.3	13	16
3	58	0.1	14	16
7	45	0.5	18	16
8	61	0.8	15	16
11	48	0.7	20	16

Le gradient boosting : itérations

Dans l'étape t , on commence par calculer les **pseudo-résidus** $r_i^{(t)}$ entre la prédiction précédente $\hat{y}_i^{(t-1)}$ et nos vraies valeurs y_i . Ils se définissent comme :

$$r_i^{(t)} = -\frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$$

Ce qui donne $r_i^{(t)} = y_i - \hat{y}_i^{(t-1)}$ pour l'erreur carrée.

X_1	X_2	X_3	Y	$\hat{y}^{(0)}$	$r_i^{(1)}$
2	51	0.3	13	16	-3
3	58	0.1	14	16	-2
7	45	0.5	18	16	2
8	61	0.8	15	16	-1
11	48	0.7	20	16	4

Le gradient boosting : itérations

On va maintenant construire un **arbre de régression, de taille limitée, qui prédit nos pseudo-résidus**. Ces prédictions sont notées $\hat{r}_i^{(t)}$. La **nouvelle estimation de l'itération** est alors :

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \nu \hat{r}_i^{(t)}$$

où $0 < \nu \leq 1$ est le **taux d'apprentissage**.

X_1	X_2	X_3	Y	$\hat{y}_i^{(0)}$	$r_i^{(1)}$	$\hat{r}_i^{(1)}$	$\hat{y}_i^{(1)}$
2	51	0.3	13	16	-3	-2	15
3	58	0.1	14	16	-2	-2	15
7	45	0.5	18	16	2	3	17.5
8	61	0.8	15	16	-1	-2	15
11	48	0.7	20	16	4	3	17.5

avec un arbre de profondeur 1 et $\nu = 0.5$

On répète ces itérations m fois.

Le gradient boosting : prédiction

Une fois nos arbres $1, \dots, m$ entraînés, ils prédisent chacun **un des pseudo-résidus** $\hat{r}_i^{(t)}$. Une **prédiction** $\hat{y}_i$ est obtenue avec :

$$\hat{y}_i = \hat{y}^{(0)} + \nu \sum_{t=1}^m \hat{r}_i^{(t)}$$

X_1	X_2	X_3	Y	$\hat{y}^{(0)}$	$\hat{r}_i^{(1)}$	$\hat{r}_i^{(2)}$	$\hat{r}_i^{(3)}$	$\hat{r}_i^{(4)}$	$\hat{r}_i^{(5)}$	$\hat{y}_i$
2	51	0.3	13	16	-2	-0.63	-1.19	-1.09	-0.33	13.38
3	58	0.1	14	16	-2	-0.63	-1.19	0.27	-0.33	14.06
7	45	0.5	18	16	3	-0.63	0.79	0.27	0.49	17.97
8	61	0.8	15	16	-2	-0.63	0.79	0.27	-0.33	15.05
11	48	0.7	20	16	3	2.5	0.79	0.27	0.49	19.53

avec des arbres de profondeur 1 et $\nu = 0.5$

Demo : https://arogozhnikov.github.io/2016/06/24/gradient_boosting_explained.html

Le gradient boosting : classification

Dans le cadre d'une **classification**, l'algorithme devient plus complexe. Si l'on considère une variable binaire y_i à estimer avec une probabilité $\hat{p}_i$, on devra **jongler entre les représentations en probabilité $\hat{p}_i$ ou en log-odd $\hat{\sigma}_i = \log(\hat{p}_i / (1 - \hat{p}_i))$** .

- La perte est l'**entropie croisée** $L(y, \hat{p}) = -y \log(\hat{p}) - (1 - y) \log(1 - \hat{p})$.
- Les **estimations se font en log-odds**, avec $\hat{\sigma}^{(0)} = \log(\sum_i y_i / \sum_i (1 - y_i))$.
- Les **résidus s'expriment en probabilité**, avec $r_i^{(t)} = y_i - \hat{p}^{(t)}$.
- les arbres sont entraînés pour séparer les $r_i^{(t)}$, mais on va attribuer une **correction aux log-odds à chaque feuille F** , avec $\hat{\delta}_F^{(t)} = \frac{\sum_{i \in F} r_i^{(t)}}{\sum_{i \in F} \hat{p}^{(t-1)}(1 - \hat{p}^{(t-1)})}$.
- La **prédiction finale se fera sur les log-odds** avec :

$$\hat{\sigma}_i = \hat{\sigma}^{(0)} + \nu \sum_{t=1}^m \hat{\delta}_{F_{it}}^{(t)}$$

où $\hat{\delta}_{F_{it}}^{(t)}$ est la correction attribuée par la feuille F_{it} contenant i à l'étape t .